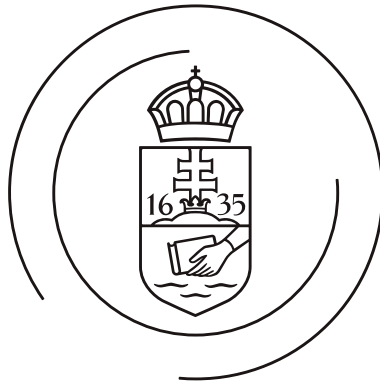




Eötvös Loránd University

ELTE 1

1 Centroid Decomposition

```

1 struct node{
2     vector<int> to;
3     vector<pair<int, int>> p; // csak ha kell
4     int sz = 0;
5     bool vis = false;
6 };
7 vector<node> g;
8 int get_sz(int x, int p = -1){
9     g[x].sz = 1;
10    for(int y : g[x].to) if(y != p && !g[y].vis) g[x].sz += get_sz(y, x);
11    return g[x].sz;
12 }
13 pair<int, int> get_c(int x, int n, int p = -1) { // elotte: get_sz
14     for(int y : g[x].to) if(y != p && !g[y].vis && g[y].sz * 2 >= n) return g[y].sz * 2 == n ? make_pair(x,
15     → y) : get_c(y, n, x);
16     return make_pair(x, x);
17 }
18 void dfs_sub(int x, int c, int d = 0, int p = -1){
19     g[x].p.emplace_back(c, d);
20     for(int y : g[x].to) if(y != p && !g[y].vis) dfs_sub(y, c, d + 1, x);
21 }
22 void centroid_decomp(int c){
23     int sz = get_sz(c);
24     c = get_c(c, sz).first;
25     g[c].vis = true;
26     dfs_sub(c, c); // centroid szülok távolsága a csúcstól (önmagát is beleértve) / a sz itt már nem jó újra
27     → kell számolni
28     // calc
29     for(int y : g[c].to) if(!g[y].vis) centroid_decomp(y);
30 }

```

2 Heavy Light Decomposition

```

1 struct node{
2     vector<int> to;
3     int l, r, i, p, hld_p, sz, d; // l: st bal, r: st jobb, i: st idx, p: os, hld_p: light edge elotti os,
4     → sz: részfa mérete, d: gyökértől vett távolság
5     // heavy út: [l, r], részfa: [i, i + sz), !!! szegmensfában g[x].i-t kell használni
6 };
7 vector<node> g;
8 int dfs_sz(int x, int d = 0, int p = -1){
9     g[x].sz = 1;
10    g[x].d = d;
11    for(int y : g[x].to) if(y != p) g[x].sz += dfs_sz(y, d + 1, x);
12    return g[x].sz;
13 }
14 int IDX = 0; // reset
15 int dfs_hld(int x, int hld_p, int p = -1){ // x = hld_p = root
16     g[x].i = g[x].r = IDX++;
17     g[x].l = g[hld_p].i;
18     g[x].p = p;
19     g[x].hld_p = hld_p;
20     sort(g[x].to.begin(), g[x].to.end(), [](int i, int j){ return g[i].sz > g[j].sz; });
21     bool fst = true;
22     for(int y : g[x].to){
23         if(y == p) continue;
24         if(fst) { g[x].r = dfs_hld(y, hld_p, x); fst = false; }
25         else dfs_hld(y, y, x);
26     }
27     return g[x].r;
28 }
29 void build_hld(int root) { dfs_sz(root); IDX = 0; dfs_hld(root, root); }

```

3 2 SAT

```

1 const int c=1000005;
2 int n, m, compdb, comp[c], res[c];
3 vector<int> el[c], el2[c];
4 bool vis[c], vis2[c];
5 vector<int> sor;
6 int par(int a) {
7     return (a<=n ? a+n : a-n);
8 }
9 void add(int a, int b) {
10     // a->b;
11     el[a].push_back(b);
12     el2[b].push_back(a);
13     a=par(a), b=par(b);
14     el[b].push_back(a);
15     el2[a].push_back(b);
16 }
17 void dfs(int a) {
18     vis[a]=true;
19     for (auto x:el[a]) {
20         if (!vis[x]) {
21             dfs(x);

```

```

22     }
23 }
24 sor.push_back(a);
25 }
26 void dfs2(int a, int s) {
27     vis2[a]=true;
28     comp[a]=s;
29     for (auto x:el2[a]) {
30         if (!vis2[x]) {
31             dfs2(x, s);
32         }
33     }
34 }
35 bool solve() {
36     // 1-től n-ig vannak a változók
37     // false ha nincs megoldás egyebkent res[i]=1 ha az i. változó igaz egy lehetséges megoldásban
38     for (int i=1; i<=2*n; i++) {
39         if (!vis[i]) {
40             dfs(i);
41         }
42     }
43     reverse(sor.begin(), sor.end());
44     for (auto x:sor) {
45         if (!vis2[x]) {
46             dfs2(x, ++compdb);
47         }
48     }
49     for (int i=1; i<=n; i++) {
50         int a=i, b=par(i);
51         if (comp[a]==comp[b]) {
52             return false;
53         }
54         if (comp[b]>comp[a]) {
55             res[i]=1;
56         }
57     }
58     return true;
59 }
60 }

```

4 Bipartite Max Matching

```

1 using namespace std;
2
3 struct HopcroftKarp {
4     std::vector<int> G, L, R;
5     int flow;
6     HopcroftKarp(int n, int m, const std::vector<std::array<int, 2>> &edges) : G(edges.size()), L(n, -1),
    ↪ R(m, -1), flow(0) {
7         std::vector<int> deg(n + 1), a, p, q(n);
8         for (auto &[x, y] : edges) { deg[x]++; }
9         for (int i = 1; i <= n; i++) { deg[i] += deg[i - 1]; }
10        for (auto &[x, y] : edges) { G[--deg[x]] = y; }
11        while (true) {
12            a.assign(n, -1), p.assign(n, -1);
13            int t = 0;
14            for (int i = 0; i < n; i++) {
15                if (L[i] == -1) {
16                    q[t++] = a[i] = p[i] = i;
17                }
18            }
19            bool match = false;
20            for (int i = 0; i < t; i++) {
21                int x = q[i];
22                if (L[a[x]] != -1) {
23                    continue;
24                }
25                for (int j = deg[x]; j < deg[x + 1]; j++) {
26                    int y = G[j];
27                    if (R[y] == -1) {
28                        while (y != -1) {
29                            R[y] = x, std::swap(L[x], y), x = p[x];
30                        }
31                        match = true, flow++;
32                        break;
33                    }
34                    if (p[R[y]] == -1) {
35                        q[t++] = y = R[y], p[y] = x, a[y] = a[x];
36                    }
37                }
38            }
39            if (!match) {
40                break;
41            }
42        }
43    }
44
45    std::vector<std::array<int, 2>> get_edges() {
46        std::vector<std::array<int, 2>> res;
47        for (int i = 0; i < L.size(); i++) {
48            if (L[i] != -1) {
49                res.push_back({i, L[i]});
50            }
51        }
52        return res;
53    }
54 }

```

54 };

5 Max Matching

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Matching {
5     queue<int> q; int ans, n;
6     vector<int> fa, s, v, pre, match;
7     Matching(auto &&g) : ans(0), n(g.size()), fa(n + 1), s(n + 1), v(n + 1), pre(n + 1, n), match(n + 1, n) {
8         for (int x = 0; x < n; ++x) if (match[x] == n) ans += Bfs(g, x, n);
9     }
10    int Find(int u) {
11        return u == fa[u] ? u : fa[u] = Find(fa[u]);
12    }
13    int LCA(int x, int y, int n) {
14        static int tk = 0; tk++; x = Find(x); y = Find(y);
15        for (; swap(x, y) if (x != n) {
16            if (v[x] == tk) return x;
17            v[x] = tk;
18            x = Find(pre[match[x]]);
19        }
20    }
21    void Blossom(int x, int y, int l) {
22        for (; Find(x) != l; x = pre[y]) {
23            pre[x] = y, y = match[x];
24            if (s[y] == 1) q.push(y), s[y] = 0;
25            for (int z: {x, y}) if (fa[z] == z) fa[z] = l;
26        }
27    }
28    bool Bfs(auto &&g, int r, int n) {
29        iota(all(fa), 0); ranges::fill(s, -1);
30        q = queue<int>(); q.push(r); s[r] = 0;
31        for (; !q.empty(); q.pop()) {
32            for (int x = q.front(); int u : g[x])
33                if (s[u] == -1) {
34                    if (pre[u] = x, s[u] = 1, match[u] == n) {
35                        for (int a = u, b = x, last;
36                             b != n; a = last, b = pre[a])
37                            last = match[b], match[b] = a, match[a] = b;
38                        return true;
39                    }
40                    q.push(match[u]); s[match[u]] = 0;
41                } else if (!s[u] && Find(u) != Find(x)) {
42                    int l = LCA(u, x, n);
43                    Blossom(x, u, l); Blossom(u, x, l);
44                }
45        }
46        return false;
47    }; // init: vector<vector<int>> gráf (n: gráf mérete), párosítás mérete: ans, párosítása i-nek: nincs ->
48    ↪ match[i] == n | van -> match[i]

```

6 Max Weighted Matching

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace weighted_blossom_tree{
5     #define d(x) (lab[x.u]+lab[x.v]-e[x.u][x.v].w*2)
6     const int N=403*2; using ll = long long; using T = int; // sum of weight, single weight
7     const T inf=numeric_limits<T>::max()-1;
8     struct Q{ int u, v; T w; } e[N][N]; vector<int> p[N];
9     int n, m=0, id, h, t, lk[N], sl[N], st[N], f[N], b[N][N], s[N], ed[N], q[N]; T lab[N];
10    void upd(int u, int v){ if(!sl[v] || d(e[u][v]) < d(e[sl[v]][v])) sl[v] = u; }
11    void ss(int v){
12        sl[v]=0; for(auto u=1; u<=n; u++) if(e[u][v].w > 0 && st[u] != v && !s[st[u]]) upd(u, v);
13    }
14    void ins(int u){ if(u <= n) q[++t] = u; else for(auto v : p[u]) ins(v); }
15    void mdf(int u, int w){ st[u]=w; if(u > n) for(auto v : p[u]) mdf(v, w); }
16    int gr(int u,int v){
17        if((v=find(p[u].begin(), p[u].end(), v) - p[u].begin()) & 1){
18            reverse(p[u].begin()+1, p[u].end()); return (int)p[u].size() - v;
19        }
20        return v;
21    }
22    void stm(int u, int v){
23        lk[u] = e[u][v].v;
24        if(u <= n) return; Q w = e[u][v];
25        int x = b[u][w.u], y = gr(u,x);
26        for(auto i=0; i<y; i++) stm(p[u][i], p[u][i+1]);
27        stm(x, v); rotate(p[u].begin(), p[u].begin()+y, p[u].end());
28    }
29    void aug(int u, int v){
30        int w = st[lk[u]]; stm(u, v); if(!w) return;
31        stm(w, st[f[w]]); aug(st[f[w]], w);
32    }
33    int lca(int u, int v){
34        for(++id; u|v; swap(u, v)){
35            if(!u) continue; if(ed[u] == id) return u;

```

```

36         ed[u] = id; if(u = st[lk[u]]) u = st[f[u]]; // not ==
37     }
38     return 0;
39 }
40 void add(int u, int a, int v){
41     int x = n+1; while(x <= m && st[x]) x++;
42     if(x > m) m++;
43     lab[x] = s[x] = st[x] = 0; lk[x] = lk[a];
44     p[x].clear(); p[x].push_back(a);
45     for(auto i=u, j=0; i!=a; i=st[f[j]]) p[x].push_back(i), p[x].push_back(j=st[lk[i]]), ins(j);
46     reverse(p[x].begin()+1, p[x].end());
47     for(auto i=v, j=0; i!=a; i=st[f[j]]) p[x].push_back(i), p[x].push_back(j=st[lk[i]]), ins(j);
48     mdf(x, x); for(auto i=1; i<=m; i++) e[x][i].w = e[i][x].w = 0;
49     memset(b[x]+1, 0, n*sizeof b[0][0]);
50     for(auto u : p[x]){
51         for(v=1; v<=m; v++) if(!e[x][v].w || d(e[u][v]) < d(e[x][v])) e[x][v] =
→ e[u][v], e[v][x] = e[v][u];
52         for(v=1; v<=n; v++) if(b[u][v]) b[x][v] = u;
53     }
54     ss(x);
55 }
56 void ex(int u){ // s[u] == 1
57     for(auto x : p[u]) mdf(x, x);
58     int a = b[u][e[u][f[u]].u], r = gr(u, a);
59     for(auto i=0; i<r; i+=2){
60         int x = p[u][i], y = p[u][i+1];
61         f[x] = e[y][x].u; s[x] = 1; s[y] = 0; sl[x] = 0; ss(y); ins(y);
62     }
63     s[a] = 1; f[a] = f[u];
64     for(auto i=r+1; i<p[u].size(); i++) s[p[u][i]] = -1, ss(p[u][i]);
65     st[u] = 0;
66 }
67 bool on(const Q &e){
68     int u=st[e.u], v=st[e.v], a;
69     if(s[v] == -1) f[v] = e.u, s[v] = 1, a = st[lk[v]], sl[v] = sl[a] = s[a] = 0, ins(a);
70     else if(!s[v]){
71         a = lca(u, v); if(!a) return aug(u,v), aug(v,u), true; else add(u,a,v);
72     }
73     return false;
74 }
75 bool bfs(){
76     memset(s+1, -1, m*sizeof s[0]); memset(sl+1, 0, m*sizeof sl[0]);
77     h = 1; t = 0; for(auto i=1; i<=m; i++) if(st[i] == i && !lk[i]) f[i] = s[i] = 0, ins(i);
78     if(h > t) return 0;
79     while(true){
80         while(h <= t){
81             int u = q[h++];
82             if(s[st[u]] != 1) for(auto v=1; v<=n; v++) if(e[u][v].w > 0 && st[u] !=
→ st[v])
83                 if(d(e[u][v])) upd(u, st[v]); else if(on(e[u][v])) return true;
84         }
85         x = inf;
86         for(auto i=n+1; i<=m; i++) if(st[i] == i && s[i] == 1) x = min(x, lab[i]>>1);
87         for(auto i=1; i<=m; i++) if(st[i] == i && sl[i] && s[i] != 1) x = min(x,
→ d(e[sl[i]][i])>>s[i]+1);
88         for(auto i=1; i<=n; i++) if(~s[st[i]]) if((lab[i] += (s[st[i]]*2-1)*x) <= 0) return
→ false;
89         for(auto i=n+1; i<=m; i++) if(st[i] == i && ~s[st[i]]) lab[i] += (2-s[st[i]]*4)*x;
90         h = 1; t = 0;
91         for(auto i=1; i<=m; i++) if(st[i] == i && sl[i] && st[sl[i]] != i &&
→ !d(e[sl[i]][i]) && on(e[sl[i]][i])) return true;
92         for(auto i=n+1; i<=m; i++) if(st[i] == i && s[i] == 1 && !lab[i]) ex(i);
93     }
94     return 0;
95 }
96 template<typename TT> pair<ll, vector<array<int, 2>>> run(int N, vector<tuple<int,int,TT>> edges){
→ // 1-based
97     for(auto &[u, v, w]: edges) ++ u, ++ v;
98     memset(ed+1, 0, m*sizeof ed[0]); memset(lk+1, 0, m*sizeof lk[0]);
99     n = m = N; id = 0; iota(st+1, st+n+1, 1); T wm = 0; ll weight = 0;
100     for(auto i=1; i<=n; i++) for(auto j=1; j<=n; j++) e[i][j] = {i,j,0};
101     for(auto [u,v,w] : edges) wm = max(wm, e[v][u].w=e[u][v].w=max(e[u][v].w,(T)w));
102     for(auto i=1; i<=n; i++) p[i].clear();
103     for(auto i=1; i<=n; i++) for(auto j=1; j<=n; j++) b[i][j] = i*(i==j);
104     fill_n(lab+1, n, wm); while(bfs());
105     vector<array<int, 2>> matching;
106     for(auto i=1; i<=n; i++) if(i < lk[i]) weight += e[i][lk[i]].w, matching.push_back({i - 1,
→ lk[i] - 1});
107     return {weight, matching};
108 }
109 #undef d
110 } // call: weighted_blossom_tree::run(n, edges) / returns: pair{weight, vector{edge}}
```

7 Flow

```

1 struct FlowEdge {
2     int v, u;
3     long long cap, flow = 0;
4     FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
5 };
6
7 struct Dinic {
8     const long long flow_inf = 1e18;
```

```

9   vector<FlowEdge> edges;
10  vector<vector<int>>> adj;
11  int n, m = 0;
12  int s, t;
13  vector<int> level, ptr;
14  queue<int> q;
15  Dinic(int n, int s, int t) : n(n), s(s), t(t), adj(n), level(n), ptr(n) {}
16  void add_edge(int v, int u, long long cap) {
17      edges.emplace_back(v, u, cap);
18      edges.emplace_back(u, v, 0);
19      adj[v].push_back(m);
20      adj[u].push_back(m + 1);
21      m += 2;
22  }
23  bool bfs() {
24      while (!q.empty()) {
25          int v = q.front();
26          q.pop();
27          for (int id : adj[v]) {
28              if (edges[id].cap - edges[id].flow < 1 || level[edges[id].u] != -1) continue;
29              level[edges[id].u] = level[v] + 1;
30              q.push(edges[id].u);
31          }
32      }
33      return level[t] != -1;
34  }
35  long long dfs(int v, long long pushed) {
36      if (pushed == 0) return 0;
37      if (v == t) return pushed;
38      for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
39          int id = adj[v][cid];
40          int u = edges[id].u;
41          if (level[v] + 1 != level[u] || edges[id].cap - edges[id].flow < 1)
42              continue;
43          long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
44          if (tr == 0) continue;
45          edges[id].flow += tr;
46          edges[id ^ 1].flow -= tr;
47          return tr;
48      }
49      return 0;
50  }
51  long long flow() {
52      long long f = 0;
53      while (true) {
54          fill(level.begin(), level.end(), -1);
55          level[s] = 0;
56          q.push(s);
57          if (!bfs()) break;
58          fill(ptr.begin(), ptr.end(), 0);
59          while (long long pushed = dfs(s, flow_inf)) f += pushed;
60      }
61      return f;
62  }
63  }
64  };

```

8 Min Cost Max Flow

```

1  struct Edge { int from, to, capacity, cost; };
2  vector<vector<int>>> adj, cost, capacity;
3  const int INF = 1e9;
4  void shortest_paths(int n, int v0, vector<int>& d, vector<int>& p) {
5      d.assign(n, INF);
6      d[v0] = 0;
7      vector<bool> inq(n, false);
8      queue<int> q;
9      q.push(v0);
10     p.assign(n, -1);
11     while (!q.empty()) {
12         int u = q.front();
13         q.pop();
14         inq[u] = false;
15         for (int v : adj[u]) {
16             if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v]) {
17                 d[v] = d[u] + cost[u][v];
18                 p[v] = u;
19                 if (!inq[v]) {
20                     inq[v] = true;
21                     q.push(v);
22                 }
23             }
24         }
25     }
26 }
27 }
28 }
29 }
30 }
31 }
32 int min_cost_flow(int N, vector<Edge> edges, int K, int s, int t) {
33     adj.assign(N, vector<int>());
34     cost.assign(N, vector<int>(N, 0));
35     capacity.assign(N, vector<int>(N, 0));

```

```

36 for (Edge e : edges) {
37     adj[e.from].push_back(e.to);
38     adj[e.to].push_back(e.from);
39     cost[e.from][e.to] = e.cost;
40     cost[e.to][e.from] = -e.cost;
41     capacity[e.from][e.to] = e.capacity;
42 }
43
44 int flow = 0;
45 int cost = 0;
46 vector<int> d, p;
47 while (flow < K) {
48     shortest_paths(N, s, d, p);
49     if (d[t] == INF)
50         break;
51     // find max flow on that path
52     int f = K - flow;
53     int cur = t;
54     while (cur != s) {
55         f = min(f, capacity[p[cur]][cur]);
56         cur = p[cur];
57     }
58     // apply flow
59     flow += f;
60     cost += f * d[t];
61     cur = t;
62     while (cur != s) {
63         capacity[p[cur]][cur] -= f;
64         capacity[cur][p[cur]] += f;
65         cur = p[cur];
66     }
67 }
68 }
69
70 if (flow < K)
71     return -1;
72 else
73     return cost;
74 }
75 }

```

9 Convex Hull Trick

```

1 const long long INF = 2e18 + 10; // Elég nagyoknak kell lennie
2 struct line{ // a * x + b / a, b: kezdetben a legrosszabb egyenes
3     mutable long long a = 0, b = INF;
4     mutable long double lef = -2e18; bool point = false; // csak a set cht-hoz
5     long long get(long long x) const { return a * x + b; }
6     long double intersect(const line& e) const { return (long double)(e.b - b) / (a - e.a); }
7     bool bad() const { return b == INF; } // ellenorzi, hogy az egyenes mindennél rosszabb-e (nincsen)
8 };
9
10 struct li_chao{ // update(line) hozzáad egy egyenest, query(x) x-helyen lévő minimum y értéket adja vissza
11     struct node{ // Elore foglalt memóriával gyorsabb
12         line e;
13         node *l = NULL, *r = NULL;
14     } *root;
15     long long L, R;
16     li_chao(long long L, long long R) : root(new node()), L(L), R(R) {}
17     void update(node* &p, long long l, long long r, line e){
18         if(e.bad()) return;
19         if(!p) p = new node();
20         int m = (l + r) / 2;
21         bool lef = e.get(L) < p->e.get(L);
22         bool mid = e.get(m) < p->e.get(m);
23         if(mid) swap(e, p->e);
24         if(r - l == 1) return;
25         else if(lef != mid) update(p->l, l, m, e);
26         else update(p->r, m, r, e);
27     }
28     void update(line e) { update(root, L, R, e); }
29     long long query(node *p, long long l, long long r, long long x) {
30         if(!p) return INF;
31         int m = (l + r) / 2;
32         if(x < m) return min(p->e.get(x), query(p->l, l, m, x));
33         return min(p->e.get(x), query(p->r, m, r, x));
34     }
35     long long query(long long x) { return query(root, L, R, x); }
36 };
37
38 struct CHT{
39     struct comp{ bool operator()(const line& e1, const line& e2) const { return !e1.point && !e2.point ?
40     ↪ e1.a > e2.a : e1.lef < e2.lef; } };
41     set<line, comp> lines;
42     static inline bool check(const line& a, const line& b, const line& c) { return a.intersect(c) <
43     ↪ a.intersect(b); }
44     void update(const line& e){
45         auto it = lines.insert(e).first;
46         if(it->b < e.b) return;
47         it->b = e.b;
48         auto prv = it == lines.begin() ? lines.end() : prev(it);
49         auto nxt = next(it);
50         if(prv != lines.end() && nxt != lines.end() && check(*prv, *it, *nxt)) {
51             lines.erase(it);
52             return;
53         }
54     }
55 }

```



```

52 while(prv != lines.end() && prv != lines.begin()){
53     auto prv2 = prev(prv);
54     if(check(*prv2, *prv, *it)){
55         lines.erase(prv);
56         prv = prv2;
57     } else {
58         break;
59     }
60 }
61 while(nxt != lines.end() && next(nxt) != lines.end()){
62     auto nxt2 = next(nxt);
63     if(check(*it, *nxt, *nxt2)) {
64         lines.erase(nxt);
65         nxt = nxt2;
66     } else{
67         break;
68     }
69 }
70 if(prv != lines.end()) it->lef = prv->intersect(*it);
71 if(nxt != lines.end()) nxt->lef = it->intersect(*nxt);
72 }
73 long long query(long long x){
74     line tmp;
75     tmp.lef = x;
76     tmp.point = true;
77     auto it = lines.upper_bound(tmp);
78     assert(it != lines.begin());
79     return prev(it)->get(x);
80 }
81 };

```

10 Float Geometry

```

1 const long double EPS = 1e-9;
2
3 struct point{
4     long double x, y;
5     point operator+(const point& p) const { return point{x + p.x, y + p.y}; }
6     point operator-(const point& p) const { return point{x - p.x, y - p.y}; }
7     point operator*(long double t) const { return point{x * t, y * t}; }
8     long double len() const { return hypot(x, y); }
9     point normalized() const { return (*this) * (1.0 / len()); }
10    bool operator<(const point& p) const { return x < p.x - EPS || (abs(x - p.x) < EPS && y < p.y - EPS); }
11 };
12
13 inline long double dot(const point& a, const point& b) { return a.x * b.x + a.y * b.y; }
14 inline long double cross(const point& a, const point& b) { return a.x * b.y - a.y * b.x; }
15 inline long double det(long double a, long double b, long double c, long double d) { return a * c - b * d; }
16 inline long double sqr(long double x) { return x*x; }
17 inline int sgn(auto x) { return (x > 0) - (x < 0); }
18 inline int dir(const point& a, const point& b, const point& c) { return sgn(cross(b - a, c - a)); }
19
20 struct line{ // a * x + b * y + c = 0, normalizáltnak kell lennie
21     long double a, b, c;
22     line(long double a_, long double b_, long double c_) : a(a_), b(b_), c(c_) {
23         long double len = hypot(a, b);
24         if(len > EPS) a /= len, b /= len, c /= len;
25     }
26     line(const point& p1, const point& p2) {
27         a = p1.y - p2.y;
28         b = p2.x - p1.x;
29         c = -a * p1.x - b * p1.y;
30         long double len = hypot(a, b);
31         if(len > EPS) a /= len, b /= len, c /= len;
32     }
33     long double dist(const point& p) { return a * p.x + b * p.y + c; }
34 };
35
36 inline bool paralell(const line& l1, const line& l2) { return abs(det(l1.a, l1.b, l2.a, l2.b)) < EPS; }
37
38 inline bool equivalent(const line& l1, const line& l2) {
39     return abs(det(l1.a, l1.b, l2.a, l2.b)) < EPS
40         && abs(det(l1.a, l1.c, l2.a, l2.c)) < EPS
41         && abs(det(l1.b, l1.c, l2.b, l2.c)) < EPS;
42 }
43
44 inline bool intersect1(long double a1, long double a2, long double b1, long double b2){
45     return max(min(a1, a2), min(b1, b2)) <= min(max(a1, a2), max(b1, b2)) + EPS;
46 }
47
48 inline bool betw(double l, double r, double x) {
49     return min(l, r) <= x + EPS && x <= max(l, r) + EPS;
50 }
51
52 bool intersect(const line& l1, const line& l2, point& res) {
53     long double zn = det(l1.a, l1.b, l2.a, l2.b);
54     if (abs(zn) < EPS) return false; // párhuzamos
55     res.x = -det(l1.c, l1.b, l2.c, l2.b) / zn;
56     res.y = -det(l1.a, l1.c, l2.a, l2.c) / zn;
57     return true;
58 }
59
60 bool intersect(point a, point b, point c, point d, point& left, point& right) { // ellenorzi a metszést,
61     ↪ metszés esetén a [left, right] szakasz a metszet
62     if (!intersect1(a.x, b.x, c.x, d.x) || !intersect1(a.y, b.y, c.y, d.y))
63         return false;

```



```

64 line m(a, b);
65 line n(c, d);
66 long double zn = det(m.a, m.b, n.a, n.b);
67 if (abs(zn) < EPS) {
68     if (abs(m.dist(c)) > EPS || abs(n.dist(a)) > EPS)
69         return false;
70     if (b < a)
71         swap(a, b);
72     if (d < c)
73         swap(c, d);
74     left = max(a, c);
75     right = min(b, d);
76     return true;
77 } else {
78     left.x = right.x = -det(m.c, m.b, n.c, n.b) / zn;
79     left.y = right.y = -det(m.a, m.c, n.a, n.c) / zn;
80     return betw(a.x, b.x, left.x) && betw(a.y, b.y, left.y) &&
81         betw(c.x, d.x, left.x) && betw(c.y, d.y, left.y);
82 }
83 }
84
85 struct circle{
86     point p;
87     long double r;
88 };
89
90 vector<point> intersection(const circle& circ, const line& l){ // kör-egyenes metszéspontok
91     long double r = circ.r, a = l.a, b = l.b, c = l.c - l.a * circ.p.x - l.b * circ.p.y;
92     double x0 = -a*c/(a*a+b*b), y0 = -b*c/(a*a+b*b);
93     if (c*c > r*r*(a*a+b*b)+EPS) return {};
94     if (abs (c*c - r*r*(a*a+b*b)) < EPS) return {point{x0, y0} + circ.p};
95     double d = r*r - c*c/(a*a+b*b);
96     double mult = sqrt (d / (a*a+b*b));
97     double ax, ay, bx, by;
98     ax = x0 + b * mult;
99     bx = x0 - b * mult;
100     ay = y0 - a * mult;
101     by = y0 + a * mult;
102     return {point{ax, ay} + circ.p, point{bx, by} + circ.p};
103 }
104
105 vector<point> intersection(circle circ1, circle circ2){ // kör-kör metszéspontok
106     point origo = circ1.p;
107     circ2.p = circ2.p - origo;
108     circ1.p = {0, 0};
109     line l(-2 * circ2.p.x, -2 * circ2.p.y, sqr(circ2.p.x) + sqr(circ2.p.y) + sqr(circ1.r) - sqr(circ2.r));
110     auto tmp = intersection(circ1, l);
111     for(auto &p : tmp) p = p + origo;
112     return tmp;
113 }
114
115 void tangents (point c, double r1, double r2, vector<line> & ans) {
116     double r = r2 - r1;
117     double z = sqr(c.x) + sqr(c.y);
118     double d = z - sqr(r);
119     if (d < -EPS) return;
120     d = sqrt (abs (d));
121     line l(0, 0, 0);
122     l.a = (c.x * r + c.y * d) / z;
123     l.b = (c.y * r - c.x * d) / z;
124     l.c = r1;
125     ans.push_back (l);
126 }
127
128 vector<line> tangents (circle a, circle b) { // 2 kör közös érintői
129     vector<line> ans;
130     for (int i=-1; i<=1; i+=2)
131         for (int j=-1; j<=1; j+=2)
132             tangents (b.p-a.p, a.r*i, b.r*j, ans);
133     for (size_t i=0; i<ans.size(); ++i)
134         ans[i].c -= ans[i].a * a.p.x + ans[i].b * a.p.y;
135     return ans;
136 }

```

11 Halfplane Intersection

```

1 // Redefine epsilon and infinity as necessary. Be mindful of precision errors.
2 const long double eps = 1e-9, inf = 1e9;
3 // Basic point/vector struct.
4 struct Point {
5     long double x, y;
6     explicit Point(long double x = 0, long double y = 0) : x(x), y(y) {} // Addition, subtraction, multiply
7     ↪ by constant, dot product, cross product.
8     friend Point operator + (const Point& p, const Point& q) { return Point(p.x + q.x, p.y + q.y); }
9     friend Point operator - (const Point& p, const Point& q) { return Point(p.x - q.x, p.y - q.y); }
10    friend Point operator * (const Point& p, const long double& k) { return Point(p.x * k, p.y * k); }
11    friend long double dot(const Point& p, const Point& q) { return p.x * q.x + p.y * q.y; }
12    friend long double cross(const Point& p, const Point& q) { return p.x * q.y - p.y * q.x; }
13 };
14 // Basic half-plane struct.
15 struct Halfplane {
16     // 'p' is a passing point of the line and 'pq' is the direction vector of the line.
17     Point p, pq;
18     long double angle;

```

```

18 Halfplane() {}
19 Halfplane(const Point& a, const Point& b) : p(a), pq(b - a) { angle = atan2l(pq.y, pq.x); }
20 // Check if point 'r' is outside this half-plane.
21 // Every half-plane allows the region to the LEFT of its line.
22 bool out(const Point& r) { return cross(pq, r - p) < -eps; }
23 // Comparator for sorting.
24 bool operator < (const Halfplane& e) const { return angle < e.angle; }
25 // Intersection point of the lines of two half-planes. It is assumed they're never parallel.
26 friend Point inter(const Halfplane& s, const Halfplane& t) {
27     long double alpha = cross((t.p - s.p), t.pq) / cross(s.pq, t.pq);
28     return s.p + (s.pq * alpha);
29 }
30 };
31 // Actual algorithm
32 vector<Point> hp_intersect(vector<Halfplane>& H) {
33     Point box[4] = { // Bounding box in CCW order
34         Point(Inf, Inf),
35         Point(-Inf, Inf),
36         Point(-Inf, -Inf),
37         Point(Inf, -Inf)
38     };
39     for(int i = 0; i < 4; i++) { // Add bounding box half-planes.
40         Halfplane aux(box[i], box[(i+1) % 4]);
41         H.push_back(aux);
42     }
43     // Sort by angle and start algorithm
44     sort(H.begin(), H.end());
45     deque<Halfplane> dq;
46     int len = 0;
47     for(int i = 0; i < int(H.size()); i++) {
48         // Remove from the back of the deque while last half-plane is redundant
49         while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))) { dq.pop_back(); --len; }
50         // Remove from the front of the deque while first half-plane is redundant
51         while (len > 1 && H[i].out(inter(dq[0], dq[1]))) { dq.pop_front(); --len; }
52         // Special case check: Parallel half-planes
53         if (len > 0 && fabs1(cross(H[i].pq, dq[len-1].pq)) < eps) {
54             // Opposite parallel half-planes that ended up checked against each other.
55             if (dot(H[i].pq, dq[len-1].pq) < 0.0) return vector<Point>();
56             // Same direction half-plane: keep only the leftmost half-plane.
57             if (H[i].out(dq[len-1].p)) { dq.pop_back(); --len; }
58             else continue;
59         }
60         // Add new half-plane
61         dq.push_back(H[i]); ++len;
62     }
63     // Final cleanup: Check half-planes at the front against the back and vice-versa
64     while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2]))) { dq.pop_back(); --len; }
65     while (len > 2 && dq[len-1].out(inter(dq[0], dq[1]))) { dq.pop_front(); --len; }
66     // Report empty intersection if necessary
67     if (len < 3) return vector<Point>();
68     // Reconstruct the convex polygon from the remaining half-planes.
69     vector<Point> ret(len);
70     for(int i = 0; i+1 < len; i++) { ret[i] = inter(dq[i], dq[i+1]); }
71     ret.back() = inter(dq[len-1], dq[0]);
72     return ret;
73 }

```

12 Integer Geometry

```

1 struct point{
2     long long x, y;
3     point operator+(const point& p) const { return {x + p.x, y + p.y}; }
4     point operator-(const point& p) const { return {x - p.x, y - p.y}; }
5     point operator*(long long t) const { return {x * t, y * t}; }
6     bool operator==(const point& p) const { return x == p.x && y == p.y; }
7     long long len() const { return x * x + y * y; }
8 };
9
10 inline long long dot(const point& a, const point& b) { return a.x * b.x + a.y * b.y; }
11 inline long long cross(const point& a, const point& b) { return a.x * b.y - b.x * a.y; }
12 inline int sgn(long long x) { return (x > 0) - (x < 0); }
13 inline int dir(const point& a, const point& b, const point& c) { return sgn(cross(b - a, c - a)); }
14
15 bool comp_args(const point& a, const point& b){ // vektorok rendezése szög alapján (azon belül hossz
16     ↪ alapján)
17     bool fa = a.y > 0 || (a.y == 0 && a.x >= 0);
18     bool fb = b.y > 0 || (b.y == 0 && b.x >= 0);
19     if(fa != fb) return fa;
20     long long c = cross(a, b);
21     return c != 0 ? c > 0 : a.len() < b.len();
22 }
23
24 inline bool contains(const point& a, const point& b, const point& p){ // szakasz tartalmazza-e
25     if(dir(a, b, p) != 0) return false;
26     long long d = dot(b - a, p - a);
27     return 0 <= d && d <= (b-a).len();
28 }
29
30 inline bool intersect1(long long a1, long long a2, long long b1, long long b2){
31     return max(min(a1, a2), min(b1, b2)) <= min(max(a1, a2), max(b1, b2));
32 }

```

```

32 inline bool intersect(const point& a1, const point& a2, const point& b1, const point& b2){ // szakaszok
33   ↪ metszik-e egymást
34   if(dir(b1, a1, b2) == 0 && dir(b1, a2, b2) == 0)
35     return intersect1(a1.x, a2.x, b1.x, b2.x) && intersect1(a1.y, a2.y, b1.y, b2.y);
36   return dir(a1, a2, b1) != dir(a1, a2, b2) && dir(b1, b2, a1) != dir(b1, b2, a2);
37 }
38
39 vector<point> convex_hull(vector<point> a){ // az a pontok konvex burka, minimális pontszámmal
40   if(a.empty()) return {};
41   int pos = min_element(a.begin(), a.end(), [](const point& a, const point& b) { return a.x < b.x || (a.x
42   ↪ == b.x && a.y < b.y); }) - a.begin();
43   swap(a[0], a[pos]);
44   sort(a.begin() + 1, a.end(), [o = a[0]](const point& a, const point& b) { int d = dir(o, a, b); return d
45   ↪ == 1 || (d == 0 && (a-o).len() < (b-o).len()); });
46   vector<point> hull;
47   for(const point &p : a){
48     while(hull.size() > 1 && dir(hull[hull.size() - 2], hull[hull.size() - 1], p) != 1) hull.pop_back();
49     hull.push_back(p);
50   }
51   int j = (int)hull.size() - 2;
52   while(j > 0 && dir(hull[j], hull[j+1], hull[0]) != 1) {
53     hull.pop_back();
54     j--;
55   }
56   if(hull.size() == 2 && hull[0] == hull[1]) hull.pop_back();
57   return hull;
58 }
59
60 vector<point> minkowski_sum(vector<point> a, vector<point> b){ // a és b konvex burkok minkowski összege
61   ↪ (konvex burok, minimális pontszámmal)
62   if(a.empty() || b.empty()) return {};
63   auto comp = [](const point& a, const point& b) { return a.y < b.y || (a.y == b.y && a.x < b.x); };
64   int min_a = min_element(a.begin(), a.end(), comp) - a.begin();
65   int min_b = min_element(b.begin(), b.end(), comp) - b.begin();
66   rotate(a.begin(), a.begin() + min_a, a.end());
67   rotate(b.begin(), b.begin() + min_b, b.end());
68   a.push_back(a[0]);
69   a.push_back(a[1]);
70   b.push_back(b[0]);
71   b.push_back(b[1]);
72   vector<point> hull;
73   int i = 0, j = 0;
74   while(i < a.size() - 2 || j < b.size() - 2) {
75     hull.push_back(a[i] + b[j]);
76     long long c = cross(a[i + 1] - a[i], b[j + 1] - b[j]);
77     if(c >= 0 && i < a.size() - 2) ++i;
78     if(c <= 0 && j < b.size() - 2) ++j;
79   }
80   return hull;
81 }
82 }

```

13 Ottoman Bentley

```

1 struct seg {
2   point p, q;
3   int id;
4
5   double get_y(double x) const {
6     if (abs(p.x - q.x) < EPS)
7       return p.y;
8     return p.y + (q.y - p.y) * (x - p.x) / (q.x - p.x);
9   }
10 };
11
12 bool intersect(const seg& a, const seg& b) // same as in intersection
13 {
14   return intersect1(a.p.x, a.q.x, b.p.x, b.q.x) &&
15     intersect1(a.p.y, a.q.y, b.p.y, b.q.y) &&
16     dir(a.p, a.q, b.p) * dir(a.p, a.q, b.q) <= 0 &&
17     dir(b.p, b.q, a.p) * dir(b.p, b.q, a.q) <= 0;
18 }
19
20 bool operator<(const seg& a, const seg& b)
21 {
22   double x = max(min(a.p.x, a.q.x), min(b.p.x, b.q.x));
23   return a.get_y(x) < b.get_y(x) - EPS;
24 }
25
26 struct event {
27   double x;
28   int tp, id;
29
30   event() {}
31   event(double x, int tp, int id) : x(x), tp(tp), id(id) {}
32
33   bool operator<(const event& e) const {
34     if (abs(x - e.x) > EPS)
35       return x < e.x;
36     return tp > e.tp;
37   }
38 };

```

```

39 set<seg> s;
40 vector<set<seg>::iterator> where;
41
42 set<seg>::iterator prev(set<seg>::iterator it) {
43     return it == s.begin() ? s.end() : --it;
44 }
45
46 set<seg>::iterator next(set<seg>::iterator it) {
47     return ++it;
48 }
49
50 // meghatároz egy metszo szakaszpárt az a-ból (x koordináta szerinti legkisebb metszéspon), ezek indexével
51 → tér vissza, ha nincs akkor {-1, -1}
52 pair<int, int> solve(const vector<seg>& a) {
53     int n = (int)a.size();
54     vector<event> e;
55     for (int i = 0; i < n; ++i) {
56         e.push_back(event(min(a[i].p.x, a[i].q.x), +1, i));
57         e.push_back(event(max(a[i].p.x, a[i].q.x), -1, i));
58     }
59     sort(e.begin(), e.end());
60     s.clear();
61     where.resize(a.size());
62     for (size_t i = 0; i < e.size(); ++i) {
63         int id = e[i].id;
64         if (e[i].tp == +1) {
65             set<seg>::iterator nxt = s.lower_bound(a[id]), prv = prev(nxt);
66             if (nxt != s.end() && intersect(*nxt, a[id]))
67                 return make_pair(nxt->id, id);
68             if (prv != s.end() && intersect(*prv, a[id]))
69                 return make_pair(prv->id, id);
70             where[id] = s.insert(nxt, a[id]);
71         } else {
72             set<seg>::iterator nxt = next(where[id]), prv = prev(where[id]);
73             if (nxt != s.end() && prv != s.end() && intersect(*nxt, *prv))
74                 return make_pair(prv->id, nxt->id);
75             s.erase(where[id]);
76         }
77     }
78     return make_pair(-1, -1);
79 }
80 }
81

```

14 Power series

```

1 // POWER SERIES OPERATIONS
2 constexpr int mod = 998244353; // = 2^k * c + 1 | primitív gyöknek jónak kell lennie | 2013265921,
→ 167772161, 2113929217
3 constexpr int N = 1 << 20; // 2^l, l <= k | max N amit transzformálni lehet
4 struct mint {
5     int x;
6     constexpr inline mint(int x = 0) : x(x) {}
7     constexpr inline mint operator+(mint o) const { return x + o.x < mod ? x + o.x : x + o.x - mod; }
8     constexpr inline mint operator-(mint o) const { return x - o.x < 0 ? x - o.x + mod : x - o.x; }
9     constexpr inline mint operator*(mint o) const { return int(uint64_t(x) * o.x % mod); }
10    constexpr inline mint &operator+=(mint o) { return *this = *this + o; }
11    constexpr inline mint &operator-=(mint o) { return *this = *this - o; }
12    constexpr inline mint &operator*=(mint o) { return *this = *this * o; }
13    constexpr inline mint inv() const { return pow(mod - 2); }
14    constexpr inline mint pow(auto x) const {
15        mint a = *this; mint b = 1; for (; x; x >>= 1) { if (x & 1) { b *= a; } a *= a; } return b;
16    }
17    constexpr inline mint sqrt() const {
18        if (pow(mod >> 1).x != 1) return 0;
19        int Q = (mod - 1) >> (__countr_zero(mod-1));
20        mint x = pow((Q + 1) >> 1), y = pow(Q);
21        for (int k = __countr_zero(mod - 1) - 1; k >= 0; --k) // TODO: fix 21
22            if (y.pow(1 << k).x != 1) {
23                x *= mint(mod_primitive_root()).pow(mod >> (k + 2));
24                y *= mint(mod_primitive_root()).pow(mod >> (k + 1));
25            }
26        return min(x.x, mod - x.x);
27    }
28    static constexpr long long mod_primitive_root(){ // kiszámítja a moduló egy primitív gyökét
29        long long primes[64] = {}; int size = 0; long long p = 2, m = mod-1;
30        while(p*p <= m) { if(m % p == 0) primes[size++] = p; while(m % p == 0) m /= p; ++p; } if(m > 1)
→ primes[size++] = m;
31        for(long long i = 2; i < mod; i++) { bool ok = true; for(int j = 0; j < size; j++) ok = ok &&
→ mint(i).pow((mod - 1) / primes[j]).x != 1; if(ok) return i; }
32        return -1;
33    }
34 };
35
36 mint w[N];
37 mint invi[N + 1];
38 __attribute__((constructor)) void init() {
39     invi[1] = w[N / 2] = 1;
40     constexpr mint g = mint(mint::mod_primitive_root()).pow(mod / N);
41     for (int i = N / 2 + 1; i < N; ++i) w[i] = w[i - 1] * g;
42     for (int i = N / 2 - 1; i > 0; --i) w[i] = w[i << 1];
43     for (int i = 2; i <= N; i++) invi[i] = invi[mod % i] * (mint() - mint(mod / i));
44 }

```

```

45 void dft(mint f[], int n) { // n ketto hatvány
46     for (int k = n / 2; k; k /= 2)
47         for (int i = 0; i < n; i += k + k)
48             for (int j = 0; j < k; ++j) {
49                 mint x = f[i + j]; mint y = f[i + j + k]; f[i + j] = x + y; f[i + j + k] = (x - y) * w[k +
→ j];
50             }
51 }
52 void ift(mint f[], int n) { // n ketto hatvány
53     for (int k = 1; k < n; k *= 2)
54         for (int i = 0; i < n; i += k + k)
55             for (int j = 0; j < k; ++j) {
56                 mint x = f[i + j]; mint y = f[i + j + k] * w[k + j]; f[i + j] = x + y; f[i + j + k] = x - y;
57             }
58     mint inv = mod - (mod - 1) / n;
59     std::reverse(f + 1, f + n);
60     for (int i = 0; i < n; ++i) f[i] *= inv;
61 }
62 struct poly : std::vector<mint> { using std::vector<mint>::vector;
63     poly &add(const poly &o) { if (size() < o.size()) resize(o.size()); for (int i = 0; i < o.size(); ++i)
→ (*this)[i] += o[i]; return *this; }
64     poly &sub(const poly &o) { if (size() < o.size()) resize(o.size()); for (int i = 0; i < o.size(); ++i)
→ (*this)[i] -= o[i]; return *this; }
65     poly &mul(const poly &o) { if (size() < o.size()) resize(o.size()); for (int i = 0; i < o.size(); ++i)
→ (*this)[i] *= o[i]; return *this; }
66     poly &mul(const mint &o) { for (mint &i: *this) i *= o; return *this; }
67     poly &derivative() { for(int i = 0; i < (int)size() - 1; i++) (*this)[i] = (*this)[i + 1] * mint(i + 1);
→ pop_back(); return *this; }
68     poly &integral() { resize(size()+1); for(int i = (int)size() - 1; i > 0; i--) (*this)[i] =
→ (*this)[i-1] * inv[i]; (*this)[0] = mint(); return *this; } // lehet overflow invi mérete N !!!
69     poly copy() const { return *this; }
70     poly &resize(auto sz) { return vector::resize(sz), *this; }
71     poly &dft(int n) { return ::dft(resize(n).data(), n), *this; }
72     poly &ift(int n) { return ::ift(resize(n).data(), n), *this; }
73     poly &ins(int sz) { return insert(begin(), sz, mint()), *this; }
74     poly &del(int sz) { return erase(begin(), begin() + sz), *this; }
75     poly &reverse() { return std::reverse(begin(), end()), *this; }
76     poly pre(int sz) const { return sz < size() ? poly(begin(), begin() + sz) : copy(); }
77     poly &reduce() { while(!empty() && back().x == 0) pop_back(); return *this; }
78     poly conv(const poly &o){
79         int n = __bit_ceil(size() + o.size() - 1);
80         return copy().dft(n).mul(o.copy().dft(n)).ift(n).resize(size() + o.size() - 1);
81     }
82     poly inv() const {
83         if (front().x == 0) return {};
84         int m = size();
85         poly inv = {front().inv()};
86         for (int k = 1; k < m; k *= 2) {
87             int n = k * 2; poly a = inv.copy().dft(n), b = pre(n).dft(n);
88             inv.sub(a.copy().mul(b).ift(n).del(k).dft(n).mul(a).ift(n).resize(k).ins(k));
89         }
90         return inv.resize(m);
91     }
92     poly log() const { // res[0] = 0
93         int n = __bit_ceil(size() * 2 - 1);
94         return copy().derivative().dft(n).mul(inv().dft(n)).ift(n).integral().resize(size());
95     }
96     poly exp() const { // p[0] == 0, különben nem valid az eredmény
97         if (front().x != 0) return {};
98         int m = size();
99         poly e = {1};
100         for (int k = 1; k < m; k *= 2) {
101             int n = k * 2;
102             poly elog = e.resize(n).log(); e.dft(n*2);
103             e.add(pre(n).sub(elog).dft(n*2).mul(e)).ift(n*2).resize(n);
104         }
105         return e.resize(m);
106     }
107     poly pow(auto k) const { // k: int, long long
108         if(k == 0) return poly{1}.resize(size());
109         int j = 0;
110         while(j < size() && (*this)[j].x == 0) ++j;
111         if(j == size()) return poly{0}.resize(size());
112         mint c = (*this)[j];
113         return copy().del(j).mul(c.inv()).log().mul(mint(k % mod)).exp().mul(c.pow(k % (mod - 1))).ins(j >
→ size() / k ? (long long)size() : j * k).resize(size());
114     }
115     poly sqrt() const { // ha nem létezik akkor az eredmény: {}
116         int j = 0;
117         while(j < size() && (*this)[j].x == 0) ++j;
118         if(j == size()) return poly{0}.resize(size());
119         mint c = (*this)[j].sqrt();
120         if(c.x == 0 || j % 2 != 0) return {};
121         return copy().del(j).mul((*this)[j].inv()).resize(size() - j / 2).pow(mint(2).inv().x).mul(c).ins(j
→ / 2);
122     }
123     poly div(const poly &o) {
124         poly a = copy().reduce().reverse(), b = o.copy().reduce().reverse();
125         int m = a.size() - b.size() + 1;
126         if(a.empty() || b.empty() || a.size() < b.size()) return b.empty() ? poly{} : poly{0};
127     }
128 }

```



```

129     return a.conv(b.resize(a.size()).inv()).resize(m).reverse();
130 }
131 poly rem(const poly& o) {
132     return copy().sub(div(o).conv(o));
133 }
134 };
135
136 /*
137 poly: ugyanúgy működik, mint az std::vector
138 muveletek: +, -, * pontoknékt, derivátl, integrál, (resize, dft, ift, ins, del, reverse)
139 constans muveletek: conv, inv, log, exp, pow, sqrt, div, rem
140 */

```

15 Berlekamp Massey

```

1 const int mod = 998244353; // inverse: mod inverz
2 struct berlekamp_massey {
3     struct poly { std::vector<int> a; poly() { a.clear(); }
4         poly (std::vector<int> &a) : a (a) {}
5         int length () const { return a.size(); }
6         poly move (int d) { std::vector<int> na (d, 0);
7             na.insert (na.end (), a.begin (), a.end ());
8             return poly (na); }
9         int calc(std::vector<int> &d, int pos) { int ret = 0;
10             for (int i = 0; i < (int) a.size (); ++i) {
11                 if ((ret += 1LL * d[pos - i] * a[i] % mod) >= mod) {
12                     ret -= mod; } }
13             return ret; }
14         poly operator - (const poly &b) {
15             std::vector<int> na (std::max (this->length (), b.length ()));
16             for (int i = 0; i < (int) na.size (); ++i) {
17                 int aa = i < this->length () ? this->a[i] : 0,
18                     bb = i < b.length () ? b.a[i] : 0;
19                 na[i] = (aa + mod - bb) % mod; }
20             return poly (na); } };
21     poly operator * (const int &c, const poly &p) { // lehet hogy ez nem fordul
22         std::vector<int> na (p.length ());
23         for (int i = 0; i < (int) na.size (); ++i) {
24             na[i] = 1LL * c * p.a[i] % mod; }
25         return na; }
26     std::vector<int> solve(vector<int> a) {
27         int n = a.size (); poly s, b;
28         s.a.push_back (1), b.a.push_back (1);
29         for (int i = 0, j = -1, ld = 1; i < n; ++i) {
30             int d = s.calc(a, i); if (d) {
31                 if ((s.length () - 1) * 2 <= i) {
32                     poly ob = b; b = s;
33                     s = s - 1LL * d * inverse(ld) % mod * ob.move (i - j);
34                     j = i; ld = d;
35                 } else {
36                     s = s - 1LL * d * inverse(ld) % mod * b.move (i - j); } } }
37         return s.a; } };
38 //Bemenet: a sorozat elso n*2 tagja (n hosszú rekurzióhoz), kimenet a kepzest leiro vektor
39 struct linear_rec {
40     const int LOG = 30, MOD = 1E9 + 7; int n;
41     std::vector<int> first, trans;
42     std::vector<std::vector<int>> bin;
43     std::vector<int> add (std::vector<int> &a, std::vector<int> &b) {
44         std::vector<int> result(n * 2 + 1, 0);
45         for (int i = 0; i <= n; ++i) for (int j = 0; j <= n; ++j)
46             if ((result[i + j] += 1LL * a[i] * b[j] % MOD) >= MOD) result[i + j] -= MOD;
47         for (int i = 2 * n; i > n; --i) {
48             for (int j = 0; j < n; ++j)
49                 if ((result[i - 1 - j] += 1LL * result[i] * trans[j] % MOD) >= MOD) result[i
50 → - 1 - j] -= MOD;
51             result[i] = 0; }
52         result.erase(result.begin() + n + 1, result.end());
53         return result; }
54     linear_rec (const std::vector<int> &first, const std::vector<int> &trans) : first(first),
55 → trans(trans) {
56         n = first.size(); std::vector<int> a(n + 1, 0); a[1] = 1; bin.push_back(a);
57         for (int i = 1; i < LOG; ++i) bin.push_back(add(bin[i - 1], bin[i - 1])); }
58     int solve (int k) {
59         std::vector<int> a(n + 1, 0); a[0] = 1;
60         for (int i = 0; i < LOG; ++i) if (k >> i & 1) a = add(a, bin[i]);
61         int ret = 0;
62         for (int i = 0; i < n; ++i) if ((ret += (long long) a[i + 1] * first[i] % MOD) >= MOD) ret
63 → -= MOD;
64         return ret; } };
65 // linear_rec(a, b): a: sorozat eleje, b: kepzes / solve(k) megadja, a k-adik elemet

```

16 String algorithms I.

```

1 vector<int> prefix_function(string s) {
2     // prefix function ABAAB -> (0, 0, 1, 1, 2)
3     int n=s.size();
4     vector<int> ans(n, 0);
5     for (int i=1; i<n; i++) {

```

```

6     int ert=ans[i-1];
7     while (ert && s[i]!=s[ert]) {
8         ert=ans[ert-1];
9     }
10    if (s[i]==s[ert]) {
11        ert++;
12    }
13    ans[i]=ert;
14 }
15 return ans;
16 }
17
18 vector<int> z_function(string s) {
19     // z function ABAAB -> (0, 0, 1, 2, 0);
20     int n=s.size();
21     vector<int> ans(n, 0);
22     int l=0, r=0;
23     int lepes=0;
24     for (int i=1; i<n; i++) {
25         int len=0;
26         if (i<r) {
27             len=min(r-i, ans[i-l]);
28         }
29         while (i+len<n && s[i+len]==s[len]) {
30             lepes++;
31             len++;
32         }
33         ans[i]=len;
34         if (i+len>r) {
35             l=i, r=i+len;
36         }
37     }
38     return ans;
39 }
40
41 vector<int> find_periods(string s) {
42     // milyen hosszú prefix ismetlesevel kapható meg s
43     // ABABA -> (2, 4, 5)
44     // a teljes periodushoz (n%i==0) feltétel kell
45     int n=s.size();
46     vector<int> z=z_function(s);
47     z[0]=n;
48     vector<int> ans;
49     for (int i=1; i<n; i++) {
50         if (i+z[i]==n) {
51             ans.push_back(i);
52         }
53     }
54     ans.push_back(n);
55     return ans;
56 }
57
58 int min_rotation(string s) {
59     // mennyivel kell elcsúsztatni ABAAB -> 2
60     int n=s.size();
61     s+=s;
62     int i=0, pos=0;
63     while (i < n) {
64         int k=i, j=i+1;
65         pos=i;
66         while (j<2*n && s[k]<=s[j]) {
67             if (s[k]<s[j]) k=i;
68             else k++;
69             j++;
70         }
71         while (i<=k) {
72             i+=j-k;
73         }
74     }
75     return pos;
76     // return s.substr(pos, n);
77 }
78 }

```

17 String algorithms II.

```

1 vector<int> manacher(string s) {
2     // egy 2*n-1 hosszú vektort ad vissza, mindig az i. majd utána az i. és i+1. között induló leghosszabb
   ↪ palindromot
3     // ABAABB -> (1, 0, 3, 0, 1, 4, 1, 0, 1, 2, 1)
4     // akár a d1 (paratlan) és d2 (páros) vektor is hasznos lehet
5     int n=s.size();
6     vector<int> d1(n, 0), d2(n, 0);
7     for (int i = 0, l = 0, r = -1; i < n; i++) {
8         int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
9         while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
10             k++;
11         }
12         d1[i] = k--;
13         if (i + k > r) {
14             l = i - k;
15             r = i + k;
16         }
17     }
18     for (int i = 0, l = 0, r = -1; i < n; i++) {
19         int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
20         while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
21             k++;

```



```

22     }
23     d2[i] = k--;
24     if (i + k > r) {
25         l = i - k - 1;
26         r = i + k;
27     }
28 }
29 vector<int> ans;
30 for (int i=0; i<n; i++) {
31     if (i) ans.push_back(2*d2[i]);
32     ans.push_back(2*d1[i]-1);
33 }
34 return ans;
35 }
36
37 vector<int> sort_cyclic_shifts(string const& s) {
38     // ABAAB -> (2, 0, 3, 1, 4)
39     int n = s.size();
40     const int alphabet = 256;
41
42     vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
43     for (int i = 0; i < n; i++)
44         cnt[s[i]]++;
45     for (int i = 1; i < alphabet; i++)
46         cnt[i] += cnt[i-1];
47     for (int i = 0; i < n; i++)
48         p[--cnt[s[i]]] = i;
49     c[p[0]] = 0;
50     int classes = 1;
51     for (int i = 1; i < n; i++) {
52         if (s[p[i]] != s[p[i-1]])
53             classes++;
54         c[p[i]] = classes - 1;
55     }
56
57     vector<int> pn(n), cn(n);
58     for (int h = 0; (1 << h) < n; ++h) {
59         for (int i = 0; i < n; i++) {
60             pn[i] = p[i] - (1 << h);
61             if (pn[i] < 0)
62                 pn[i] += n;
63         }
64         fill(cnt.begin(), cnt.begin() + classes, 0);
65         for (int i = 0; i < n; i++)
66             cnt[c[pn[i]]]++;
67         for (int i = 1; i < classes; i++)
68             cnt[i] += cnt[i-1];
69         for (int i = n-1; i >= 0; i--)
70             p[--cnt[c[pn[i]]]] = pn[i];
71         cn[p[0]] = 0;
72         classes = 1;
73         for (int i = 1; i < n; i++) {
74             pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
75             pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
76             if (cur != prev)
77                 ++classes;
78             cn[p[i]] = classes - 1;
79         }
80         c.swap(cn);
81     }
82     return p;
83 }
84
85 vector<int> suffix_array_construction(string s) {
86     // a suffixeket rendezzi
87     "$" mindennel kisebb
88     // ABAAB -> (2, 3, 0, 4, 1)
89     s += "$";
90     vector<int> sorted_shifts = sort_cyclic_shifts(s);
91     sorted_shifts.erase(sorted_shifts.begin());
92     return sorted_shifts;
93 }
94
95 vector<int> lcp_construction(string const& s, vector<int> const& p) {
96     // csak a masikkal együtt mukodik (ket suffix arrayben szomszedos suffix lcp-je)
97     // vector<int> res=lcp_construction(s, suffix_array_construction(s));
98     // ABAAB -> (1, 2, 0, 1)
99     int n = s.size();
100     vector<int> rank(n, 0);
101     for (int i = 0; i < n; i++)
102         rank[p[i]] = i;
103
104     int k = 0;
105     vector<int> lcp(n-1, 0);
106     for (int i = 0; i < n; i++) {
107         if (rank[i] == n - 1) {
108             k = 0;
109             continue;
110         }
111         int j = p[rank[i] + 1];
112         while (i + k < n && j + k < n && s[i+k] == s[j+k])
113             k++;
114         lcp[rank[i]] = k;
115         if (k)
116             k--;
117     }
118     return lcp;
119 }
120 }

```

18 Treap

```

1 // TREAP
2 mt19937 rnd(42123); // mt19937_64 ha long long kell
3 struct node { // az upd() és push()-t kell implementálni / upd()-et a konstruktor is hívja
4     int val, w, size; // val érték (cserélhető), w súly, size a részfa mérete
5     node *l, *r; // bal, jobb gyerek
6     node(int c) : val(c), w(rnd()), size(1), l(NULL), r(NULL) { upd(); }
7     ~node() { delete l; delete r; }
8     inline void upd() {} // update az l, r-ból
9     inline void push() {} // push l, r-be
10 } *treap;
11 int size(node *p) { return p ? p->size : 0; }
12 void split(node *p, node *&l, node *&r, int val) { // l < val / val <= r
13     if(!p) { l = r = NULL; return; }
14     p->push();
15     if(size(p->l) < val) split(p->r, p->r, r, val - size(p->l) - 1), l = p;
16     else split(p->l, l, p->l, val), r = p;
17     p->size = 1 + size(p->l) + size(p->r); p->upd();
18 }
19 void merge(node *&p, node *l, node *r) {
20     if(!l || !r) { p = l ? l : r; return; }
21     if(l->w < r->w) l->push(), merge(l->r, l->r, r), p = l;
22     else r->push(), merge(r->l, l, r->l), p = r;
23     p->size = 1 + size(p->l) + size(p->r); p->upd();
24 }

```

19 Link Cut Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 typedef struct snode *sn;
5 struct snode { ////////////// VARIABLES
6     sn p, c[2]; // parent, children
7     bool flip = 0; // subtree flipped or not
8     int size; // # splay tree csúcs, aktuálisban
9     ll val; // value in node
10     snode(int _val) : val(_val) { p = c[0] = c[1] = NULL; upd(); }
11     friend int get_size(sn x) { return x ? x->size : 0; }
12     void prop() { // lazy prop
13         if (!flip) return;
14         swap(c[0], c[1]);
15         flip = 0;
16         for (int i = 0; i < 2; i++)
17             if (c[i]) c[i]->flip ^= 1;
18     }
19     void upd() { // recalc vals
20         for (int i = 0; i < 2; i++) if (c[i]) c[i]->prop();
21         size = 1 + get_size(c[0]) + get_size(c[1]);
22         // virtuális részfa adatok használata
23     }
24     void vupd(){}
25     ////////////// SPLAY TREE OPERATIONS
26     int dir() {
27         if (!p) return -2;
28         for (int i = 0; i < 2; i++) if (p->c[i] == this) return i;
29         return -1; // p is path-parent pointer
30     } // -> not in current splay tree
31     bool is_root() { return dir() < 0; }
32     friend void set_link(sn x, sn y, int d) { if (y) y->p = x; if (d >= 0) x->c[d] = y; }
33     void rot() { // assume p and p->p propagated
34         assert(!is_root());
35         int x = dir(); sn pa = p;
36         set_link(pa->p, this, pa->dir()); set_link(pa, c[x ^ 1], x); set_link(this, pa, x ^ 1);
37         pa->upd();
38     }
39     void splay() {
40         while (!is_root() && !p->is_root()) {
41             p->p->prop(), p->prop(), prop();
42             dir() == p->dir() ? p->rot() : rot(); rot();
43         }
44         if (!is_root()) p->prop(), prop(), rot();
45         prop(); upd();
46     }
47     sn fbo(int b) { // find by order
48         prop(); int z = get_size(c[0]); // of splay tree
49         if (b == z) { splay(); return this; }
50         return b < z ? c[0]->fbo(b) : c[1]->fbo(b - z - 1);
51     }
52     ////////////// BASE OPERATIONS
53     void access() { // bring this to top of tree, propagate
54         for (sn v = this, pre = NULL; v; v = v->p) {
55             v->splay(); // now switch virtual children
56             if (pre) vupd(); // pre törlése (mostantól rendes gyerek)
57             if (v->c[1]) vupd(); // c[1] hozzáadása (mostantól virtuális gyerek)
58             v->c[1] = pre; v->upd(); pre = v;
59         }
60     }

```

```

61     splay();
62     assert(!c[1]); // right subtree is empty
63 }
64 void make_root() { // ez lesz a fa gyökere
65     access(); flip ^= 1; access();
66     assert(!c[0] && !c[1]);
67 }
68 ////////////////////////////////////////////////// QUERIES
69 friend sn lca(sn x, sn y) {
70     if (x == y) return x;
71     x->access(), y->access();
72     if (!x->p) return NULL;
73     x->splay();
74     return x->p ? x; // y was below x in latter case
75 } // access at y did not affect x -> not connected
76 friend bool connected(sn x, sn y) { return lca(x, y); }
77 int dist_root() { access(); return get_size(c[0]); } // # nodes above
78 sn get_root() { // get root of LCT component
79     access(); sn a = this;
80     while (a->c[0]) a = a->c[0], a->prop();
81     a->access();
82     return a;
83 }
84 sn get_par(int b) { // get b-th parent on path to root / can also get min, max on path to root,
→ etc
85     access(); b = get_size(c[0]) - b;
86     assert(b >= 0);
87     return fbo(b);
88 }
89 ////////////////////////////////////////////////// MODIFICATIONS
90 void set(ll v) { access(); val = v; upd(); } // changes value
91 friend void link(sn x, sn y, bool force = 0) { // ha force: x - y él minden esetben / ha nem force:
→ akkor y-nak gyökérnek kell lenni
92     assert(!connected(x, y));
93     if (force) y->make_root(); // make x par of y
94     else { y->access(); assert(!y->c[0]); }
95     x->access(); set_link(y, x, 0); y->upd();
96 }
97 friend void cut(sn y) { // cut y from its parent / ha nincs RTE
98     y->access();
99     assert(y->c[0]);
100     y->c[0]->p = y->c[0] = NULL;
101     y->upd();
102 }
103 friend void cut(sn x, sn y) { // if x, y adj in tree
104     x->make_root(); y->access();
105     assert(y->c[0] == x && !x->c[0] && !x->c[1]);
106     cut(y);
107 }
108 };
109 /*
110 Link-cut tree, muveletek: link, cut, set / lca, connected, dist_root, get_root, get_par
111 Ut query-hez a get_par-hoz hasonló implementáció kell + fbo implementáció, ha nem a teljes út kell. / Ha a,
→ b út kell: 1. make_root(a), 2. query b-tol gyökérig
112 Részfa adatokhoz a vupd()-et kell módosítani, (az upd()-ben is bele kell írni) / fontos kell legyen a
→ muveletnek inverze
113 */

```

20 Math

```

1 int gcd(int a, int b, int& x, int& y) {
2     // lehet hogy long long kell
3     // x-et es y-t beallitja ugy, hogy a*x+b*y=gcd(a, b) teljesul
4     x = 1, y = 0;
5     int x1 = 0, y1 = 1, a1 = a, b1 = b;
6     while (b1) {
7         int q = a1 / b1;
8         tie(x, x1) = make_tuple(x1, x - q * x1);
9         tie(y, y1) = make_tuple(y1, y - q * y1);
10        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
11    }
12    return a1;
13 }

```

21 Bölcsességek

- Az $O(N^2)$ $N \leq 20000$ -ig jó.
- A bitset jó bármekkora N -re.
- Fontos a precision.
- Néha enni és inni is kellene.
- Lebegőpontos számítás???

- Edge case-ek fontosak.
- Lehet, hogy az idegeskedés nem segít.